

1. Import Libraries

```
import streamlit as st
from google import genai
```

Why do we use this?

- `import` → Used to import a library.
 - `streamlit` → Used to create a web application.
 - `as st` → Gives a short name (`st`) instead of writing `streamlit` every time.
 - `genai` → Used to connect with Google's Gemini AI model.
-

2. API Key

```
API_KEY = "YOUR_GEMINI_API_KEY"
```

Why do we use this?

- API Key is a unique secret key.
 - It allows our program to access the Gemini API.
 - Without the API key, the chatbot cannot communicate with Gemini.
-

3. Create Gemini Client

```
client = genai.Client(api_key=API_KEY)
```

Why do we use this?

- Creates a connection with the Gemini API.
 - The client object is used to send questions and receive answers.
-

4. Resume Data

```
my_data = """
...
"""
```

Why do we use this?

- Stores the resume information.
 - Triple quotes (`"""`) are used for writing multiple lines of text.
 - The chatbot uses this data to answer resume-related questions.
-

5. Page Configuration

```
st.set_page_config(  
    page_title="Mukund AI Assistant",  
    page_icon="🤖",  
    layout="wide"  
)
```

Why do we use this?

- Sets the basic settings of the web page.

page_title

- Sets the browser tab title.

page_icon

- Shows an icon in the browser tab.

layout="wide"

- Makes the page use the full browser width.
-

6. Page Title

```
st.title("🤖 Mukund AI Resume Assistant")
```

Why do we use this?

- Displays a large heading on the web page.
-

7. Description

```
st.write("Ask me anything about Mukund or any general question.")
```

Why do we use this?

- Displays normal text below the title.
 - Gives instructions to the user.
-

8. Check Chat History

```
if "messages" not in st.session_state:
```

Why do we use this?

- Checks whether chat history already exists.
 - If it does not exist, a new chat history will be created.
-

9. Create Empty Chat List

```
st.session_state.messages = []
```

Why do we use this?

- Creates an empty list.
- Stores all user and bot messages.

Example

```
[]
```

10. Display Previous Messages

```
for message in st.session_state.messages:
```

Why do we use this?

- Reads every message stored in the chat history.
 - Displays old conversations.
-

11. Create Chat Bubble

```
with st.chat_message(message["role"]):
```

Why do we use this?

- Creates a chat bubble.
 - The role decides whether it is a **user** message or a **bot** message.
-

12. Show Message

```
st.markdown(message["content"])
```

Why do we use this?

- Displays the message inside the chat bubble.
-

13. Chat Input Box

```
question = st.chat_input("Type your question...")
```

Why do we use this?

- Creates an input box.
 - Allows the user to type a question.
-

14. Check User Input

if question:

Why do we use this?

- Runs the code only if the user has entered a question.
-

15. Save User Message

```
st.session_state.messages.append(  
    {"role": "user", "content": question}  
)
```

Why do we use this?

- Saves the user's message in chat history.
-

16. Show User Message

with st.chat_message("user"):

Why do we use this?

- Creates the user's chat bubble.
-

17. Display User Question

```
st.markdown(question)
```

Why do we use this?

- Displays the user's question on the screen.
-

18. Create Prompt

```
prompt = f"""  
...  
"""
```

Why do we use this?

- Creates instructions for Gemini.
- Includes:
 - Rules
 - Resume information
 - User question

f before the string allows Python to insert variables like {my_data} and {question}.

19. Try Block

try:

Why do we use this?

- Executes code safely.
 - If an error occurs, the program will not stop.
-

20. Send Request to Gemini

```
response = client.models.generate_content(
```

Why do we use this?

- Sends the prompt to Gemini.
 - Gemini processes the request and generates a response.
-

21. Select AI Model

```
model="gemini-2.5-flash"
```

Why do we use this?

- Specifies which Gemini model should answer the question.
-

22. Send Prompt

```
contents=prompt
```

Why do we use this?

- Sends the complete prompt to Gemini.
-

23. Store Response

```
answer = response.text
```

Why do we use this?

- Saves Gemini's answer into the answer variable.
-

24. Error Handling

```
except Exception as e:
```

Why do we use this?

- Catches any errors.
 - Prevents the program from crashing.
-

25. Store Error Message

```
answer = f"Error: {e}"
```

Why do we use this?

- Displays the error message if something goes wrong.
-

26. Save Bot Response

```
st.session_state.messages.append(  
    {"role": "assistant", "content": answer}  
)
```

Why do we use this?

- Saves the bot's reply in chat history.
-

27. Create Bot Chat Bubble

```
with st.chat_message("assistant"):
```

Why do we use this?

- Creates a chatbot response bubble.
-

28. Display Bot Response

```
st.markdown(answer)
```

Why do we use this?

- Displays the Gemini response on the screen.
-

Program Flow

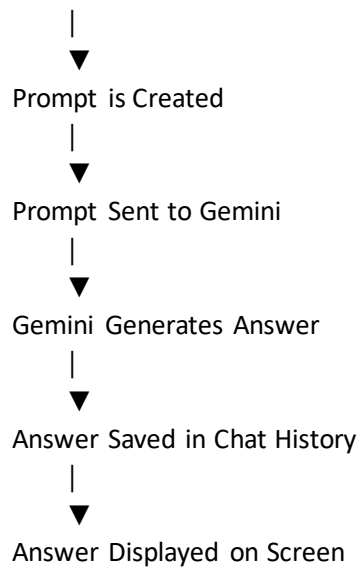
User Opens Website



Types a Question



Question is Saved



Complete Code

```
import streamlit as st
from google import genai

# -----
# Gemini API
# -----
API_KEY = "YOUR_GEMINI_API_KEY"

client = genai.Client(api_key=API_KEY)

# -----
# Your Resume Data
# -----
my_data = """
Your Resume Data Here
"""

# -----
# Page Settings
# -----
st.set_page_config(
    page_title="Mukund AI Assistant",
    page_icon="🤖",
    layout="wide"
)

st.title("🤖 Mukund AI Resume Assistant")
st.write("Ask me anything about Mukund or any general question.")
```

```

# -----
# Chat History
# -----
if "messages" not in st.session_state:
    st.session_state.messages = []

# Show old messages
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# -----
# User Input
# -----
question = st.chat_input("Type your question...")

if question:

    st.session_state.messages.append(
        {"role": "user", "content": question}
    )

    with st.chat_message("user"):
        st.markdown(question)

    prompt = f"""
You are Mukund Kumar's AI Assistant.

Rules:
1. Answer using the resume if possible.
2. Otherwise answer using general knowledge.
3. Keep answers simple.

Resume:
{my_data}

Question:
{question}
"""

    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=prompt
        )

        answer = response.text

```



```
except Exception as e:
    answer = f"Error: {e}"

st.session_state.messages.append(
    {"role": "assistant", "content": answer}
)

with st.chat_message("assistant"):
    st.markdown(answer)
```